

Python Grundlagen

Programmieren als Denkweise

- **Informatiker-Denkweise:** Kombination aus mathematischer Präzision, ingenieurmäßigem Design und wissenschaftlicher Beobachtung
- **Formale vs. natürliche Sprachen:**
 - Formale Sprachen (wie Python): eindeutig, präzise, wörtlich zu nehmen
 - Natürliche Sprachen: mehrdeutig, redundant, enthalten Metaphern und Redewendungen

Arithmetische Operatoren

Operator	Beschreibung	Beispiel	Ergebnis
+	Addition	30 + 12	42
-	Subtraktion	43 - 1	42
*	Multiplikation	6 * 7	42
/	Division (ergibt Fließkommazahl)	84 / 2	42.0
//	Ganzzahl-Division (abrunden)	85 // 2	42
**	Potenzierung	7 ** 2	49
%	Modulo (Rest nach Division)	5 % 2	1

⚠ **Vorsicht:** ^ ist KEIN Potenzierungsoperator (wie in anderen Sprachen), sondern ein bitweiser XOR-Operator

Zahlentypen

Typ	Beschreibung	Beispiel
int	Ganze Zahlen	42
float	Fließkommazahlen	42.0

Typkonvertierung

- `int(42.9)` → 42 (schneidet Dezimalstellen ab)
- `float(42)` → 42.0
- `int("126")` → 126 (wandelt String in Zahl um)
- `float("12.6")` → 12.6

Tipp: Für große Zahlen Unterstriche zur besseren Lesbarkeit verwenden: `1_000_000` statt `1000000`

Operationsreihenfolge (PEMDAS)

1. **Parenteses** (Klammern): `()`

2. **Exponents** (Potenzierung): `**`
3. **Multiplication/Division** (Multiplikation/Division): `*`, `/`, `//`, `%`
4. **Addition/Subtraction** (Addition/Subtraktion): `+`, `-`

Beispiele:

- `6 + 6 ** 2 = 6 + 36 = 42`
- `12 + 5 * 6 = 12 + 30 = 42`
- `(12 + 5) * 6 = 17 * 6 = 102`

Strings (Zeichenketten)

Operation	Beschreibung	Beispiel	Ergebnis
Erstellung	Mit einfachen oder doppelten Anführungszeichen	<code>'Hello'</code> oder <code>"Hello"</code>	<code>'Hello'</code>
<code>+</code>	Verkettung (Concatenation)	<code>'Well, ' + "it's a small" + 'world.'</code>	<code>"Well, it's a small world."</code>
<code>*</code>	Wiederholung	<code>'Spam, ' * 4</code>	<code>'Spam, Spam, Spam, Spam, '</code>
<code>len()</code>	Länge eines Strings	<code>len('Spam')</code>	<code>4</code>

Wichtig:

- Einfache (`'`) und doppelte (`"`) Anführungszeichen sind gleichwertig
- Doppelte Anführungszeichen sind nützlich für Strings mit Apostrophen: `"it's a small"`
- Typografische Anführungszeichen (`" "`) und Backticks (```) sind NICHT erlaubt und erzeugen Syntaxfehler

Funktionen

Funktion	Beschreibung	Beispiel	Ergebnis
<code>round()</code>	Rundet auf die nächste Ganzzahl	<code>round(42.4)</code>	<code>42</code>
		<code>round(42.6)</code>	<code>43</code>
<code>abs()</code>	Absoluter Wert	<code>abs(-42)</code>	<code>42</code>
<code>type()</code>	Gibt den Typ eines Werts zurück	<code>type(42)</code>	<code>int</code>
		<code>type(42.0)</code>	<code>float</code>
		<code>type('Hello')</code>	<code>str</code>
<code>len()</code>	Gibt die Länge eines Strings zurück	<code>len('Hello')</code>	<code>5</code>

Wichtig bei Funktionsaufrufen:

- Runde Klammern (`()`) sind Pflicht!
- `abs(-42)` → korrekt
- `abs -42` → Syntaxfehler

Ausdrücke und Werte

- Ein **Ausdruck** ist eine Kombination aus Operatoren und Werten
- Jeder Ausdruck hat einen **Wert**
- Jeder Wert hat einen **Typ**

Häufige Fehler und Debugging

Syntaxfehler

- Fehlende Klammern bei Funktionsaufrufen: `abs 42` statt `abs(42)`
- Falsche Anführungszeichen: ``Hello`` oder `"Hello"` statt `"Hello"` oder `'Hello'`
- Verwendung von Kommas in großen Zahlen: `1,000,000` erzeugt ein Tupel `(1, 0, 0)` statt einer Zahl

TypeError

- Tritt auf, wenn Operanden den falschen Typ haben
- Beispiel: `'126' / 3` erzeugt einen `TypeError`, weil man einen String nicht durch eine Zahl teilen kann
- Lösung: Typkonvertierung mit `int('126') / 3`

Prüfungstipps

1. **Operatoren vs. Funktionen:** Verstehe den Unterschied zwischen Operatoren (`+`, `-`, `*`, `/`) und Funktionen (`abs()`, `round()`, `len()`)
2. **Typkonvertierung:** Übe die Konvertierung zwischen Datentypen (`int()`, `float()`, `str()`)
3. **Stringoperationen:** Beachte, dass `+` und `*` bei Strings andere Bedeutungen haben als bei Zahlen
4. **Operationsreihenfolge:** Beachte die Priorität von Operatoren (PEMDAS)
5. **Syntaxregeln:** Achte auf korrekte Klammern, Anführungszeichen und Funktionsaufrufe

Mathematische Umrechnungen (Übungsbeispiele)

- Zeit: 1 Minute = 60 Sekunden
- Distanz: 1 Meile $\approx$ 1,61 Kilometer
- Geschwindigkeit: Entfernung $\div$ Zeit

Beispielberechnungen:

1. 42 Minuten und 42 Sekunden in Sekunden:

```
42 * 60 + 42 # = 2520 + 42 = 2562 Sekunden
```

2. 10 Kilometer in Meilen:

```
10 / 1.61 #  $\approx$  6.21 Meilen
```

3. Durchschnittsgeschwindigkeit für 10 km in 42:42 (Minuten: Sekunden):

```
# Zeit in Sekunden
zeit_sek = 42 * 60 + 42 # = 2562 Sekunden

# Distanz in Meilen
distanz_meilen = 10 / 1.61 # ≈ 6.21 Meilen

# Sekunden pro Meile
sek_pro_meile = zeit_sek / distanz_meilen # ≈ 412.56 Sekunden/Meile

# Minuten und Sekunden pro Meile
min_pro_meile = sek_pro_meile // 60 # = 6 Minuten
rest_sek = sek_pro_meile % 60 # ≈ 52.56 Sekunden

# Meilen pro Stunde
mph = distanz_meilen / (zeit_sek / 3600) # ≈ 8.73 mph
```